

Cheatsheet: Arrays e Objetos em JavaScript

Array e Objetos em JavaScript	Descrição	Exemplo de Código
Declaração de Array	Arrays em JavaScript são ordenados, o que significa que os elementos são armazenados em uma sequência específica.	<pre>const fruits = ["apple", "banana", "cherry"];</pre>
Indexação de Array	Os arrays são indexados a partir de zero, o que significa que o primeiro elemento está no índice 0, o segundo no índice 1, e assim por diante.	<pre>const fruits = ["apple", "banana", "cherry"]; const firstFruit = fruits[0]; // "apple" const secondFruit = fruits[1]; // "banana"</pre>
Comprimento do Array	A propriedade length é usada para determinar o número de itens presentes em um array.	<pre>const fruits = ["apple", "banana", "cherry"]; const numFruits = fruits.length; // 3 console.log(numFruits);</pre>
Mutabilidade de Array	Arrays em JavaScript são mutáveis, o que significa que você pode alterar, adicionar ou remover elementos após o array ser criado.	<pre>const fruits = ["apple", "banana", "cherry"]; fruits[2] = "strawberry"; // Modifying an element fruits[3] = "Kiwi"; // Adding an element</pre>
método push	Adiciona um ou mais elementos ao final de um array.	<pre>const fruits = ["apple", "banana"]; fruits.push("orange", "strawberry"); console.log(fruits)</pre>
método pop	Remove o último elemento de um array e o retorna.	<pre>const fruits = ["apple", "banana", "orange"]; const removedFruit = fruits.pop(); console.log('Fruits are',fruits) console.log('Removed fruits are',removedFruit)</pre>
métodos de deslocamento	Remove o primeiro elemento de um array e o retorna.	<pre>Removes the first element from an array and returns it.</pre>

método unshift	Adiciona um ou mais elementos ao início de um array e o retorna.	<pre>const fruits = ["banana", "orange"]; fruits.unshift("apple", "strawberry"); console.log(fruits);</pre>
método splice	Altera o conteúdo de um array removendo, substituindo ou adicionando elementos em uma posição especificada.	<pre>const fruits = ["apple", "banana", "cherry"]; fruits.splice(1, 1, "grape"); // Replace the second element with "grape" console.log(fruits)</pre>
método concat	O método concat em arrays JavaScript combina arrays em sequência, criando um novo array contendo os elementos dos arrays originais na ordem em que foram concatenados.	<pre>const fruits = ["apple", "banana"]; const additionalFruits = ["orange", "strawberry"]; const combinedFruits = fruits.concat(additionalFruits); console.log('combinedFruits are', combinedFruits)</pre>
método slice	Retorna uma cópia rasa de uma porção de um array em um novo array.	<pre>const fruits = ["apple", "banana", "cherry", "orange"]; const slicedFruits = fruits.slice(1, 3); // Creates a new array with elements from index 1 to 2 (not ir console.log('slicedFruits are',slicedFruits)</pre>
método indexOf	Este método é usado para encontrar o índice de um elemento especificado dentro de um array. Ele retorna o índice da primeira ocorrência do elemento no array, ou -1 se o elemento não for encontrado.	<pre>const fruits = ["apple", "banana", "cherry", "banana"]; const index = fruits.indexOf("banana"); // Returns 1 (the first occurrence of "banana") console.log('Index of banana is', index)</pre>
método reverse	O método reverse inverte a ordem dos elementos em um array, efetivamente revertendo o array no local.	<pre>const fruits = ["apple", "banana", "cherry"]; fruits.reverse(); // Reverses the order of the array console.log(fruits)</pre>
método sort	O método sort é usado para ordenar os elementos de um array no local e	<pre>const numbers = [4, 2, 8, 6, 1,10]; numbers.sort(); // Sorts as strings: [1,10, 2, 4, 6, 8] numbers.sort((a, b) => a - b); // Sorts as numbers: [1, 2, 4, 6, 8] console.log(numbers)</pre>

	retorna o array ordenado. Por padrão, ele ordena os elementos como strings e em ordem lexicográfica.	
Iteração de Array	Um loop for pode ser usado para iterar através dos elementos de um array para acessar e manipular cada item no array.	<pre>const fruits = ['apple', 'banana', 'cherry', 'date']; for (let i = 0; i < fruits.length; i++) { console.log(fruits[i]); }</pre>
forEach	O método forEach itera através de um array e aplica uma função fornecida a cada elemento.	<pre>function sendWelcomeEmail(email) { console.log(`Welcome email sent to \${email}`); } const users = [{ name: 'Alice', email: 'alice@example.com' }, { name: 'Bob', email: 'bob@example.com' }, { name: 'Charlie', email: 'charlie@example.com' },]; users.forEach((user) => { sendWelcomeEmail(user.email); });</pre>
método map	O método map cria um novo array aplicando uma função fornecida a cada elemento do array original.	<pre>const products = [{ name: 'Laptop', price: 1000 }, { name: 'Smartphone', price: 500 }, { name: 'Tablet', price: 300 },]; products.map((product) => { console.log(`The price of \${product.name} is \${product.price}`); });</pre>
método filter	O método filter cria um novo array contendo elementos que atendem a uma condição específica. É útil para extrair dados específicos de um array.	<pre>const products = [{ name: 'Laptop', price: 1000 }, { name: 'Smartphone', price: 500 }, { name: 'Tablet', price: 300 }, { name: 'Monitor', price: 250 }, { name: 'Keyboard', price: 50 },]; function filterProductsByPriceRange(products, minPrice, maxPrice) { return products.filter((product) => product.price >= minPrice && product.price <= maxPrice); } const minPrice = 100; // Minimum price threshold const maxPrice = 500; // Maximum price threshold const filteredProducts = filterProductsByPriceRange(products, minPrice, maxPrice); filteredProducts.forEach((product) => { console.log(`\${product.name} is of \${product.price}`); });</pre>
método reduce	O método reduce permite que você reduza um array a um único valor	<pre>const orderPrices = [50, 30, 25, 40, 15]; const totalOrderValue = orderPrices.reduce((total, price) => total + price, 0); console.log(`The total value of order is `, totalOrderValue)</pre>

	aplicando uma função a cada elemento. É excelente para agregar dados.	
método find	O método find retorna o primeiro elemento em um array que satisfaz uma condição especificada. É útil para buscar dados específicos.	<pre>const employees = [{ id: 1, name: 'Alice', Eid: 'EMP001', 'Contact details': 'alice@example.com', Role: 'Manager', Designation: 'Software Engineer' }, { id: 2, name: 'Bob', Eid: 'EMP002', 'Contact details': 'bob@example.com', Role: 'Engineer', Designation: 'Software Engineer' }, { id: 3, name: 'Charlie', Eid: 'EMP003', 'Contact details': 'charlie@example.com', Role: 'Analyst', Designation: 'Software Engineer' },]; const employee = employees.find((e) => e.id === 2); console.log(`Details of the employee\nname: \${employee.name}\nEid: \${employee.Eid}\nContact details: \${employee['Contact details']}\nDesignation: \${employee.Designation}`);</pre>
Array 2D	Um array 2D pode ser criado inicializando um array de arrays.	<pre>const grid = [[1, 2, 3], [4, 5, 6], [7, 8, 9]];</pre>
Acessar Array 2D	Para acessar um elemento específico em um array 2D, você precisa fornecer os índices da linha e da coluna.	<pre>for (let i = 0; i < grid.length; i++) { for (let j = 0; j < grid[i].length; j++) { console.log(`Element at (\${i}, \${j}): \${grid[i][j]}`); } }</pre>
Array 2D para reservar assento	Você pode criar um sistema de reserva usando um array 2D.	<pre><!DOCTYPE html> <html> <head> <style> /* CSS for styling the seats */ .seating-chart { display: grid; grid-template-columns: repeat(3, 70px); gap: 10px; justify-content: center; } .seat { width: 70px; height: 40px; text-align: center; line-height: 40px; border: 1px solid #ccc; cursor: pointer; } .booked { background-color: #FF0000; /* Red */ cursor: not-allowed; color: white; /* Set the text color to white for booked seats */ } .available { background-color: #7FFF00; /* Light Green */ } .select-button { width: 100%; padding: 10px; margin: 10px; background-color: #007BFF; /* Blue */ color: white; border: none; cursor: pointer; } </style> </head> <body> <h2>Movie Theater Seating</h2> <div id="seating-chart" class="seating-chart"> <div class="seat available" onclick="bookSeat(0, 0)">A1</div></pre>

		<pre><div class="seat available" onclick="bookSeat(0, 1)">A2</div> <div class="seat available" onclick="bookSeat(0, 2)">A3</div> <div class="seat available" onclick="bookSeat(1, 0)">B1</div> <div class="seat available" onclick="bookSeat(1, 1)">B2</div> <div class="seat available" onclick="bookSeat(1, 2)">B3</div> <div class="seat available" onclick="bookSeat(2, 0)">C1</div> <div class="seat available" onclick="bookSeat(2, 1)">C2</div> <div class="seat available" onclick="bookSeat(2, 2)">C3</div> </div> <button class="select-button" onclick="bookRandomSeat()">Select Random Seat</button> <script> // JavaScript for booking seats const theaterSeats = [['X', 'O', 'X'], ['O', 'X', 'O'], ['X', 'O', 'X']]; function bookSeat(row, col) { if (theaterSeats[row][col] === 'O') { theaterSeats[row][col] = 'X'; updateSeatStatus(row, col, 'booked'); alert(`Seat \${String.fromCharCode(65 + row)}\${col + 1} is booked.`); } else { alert(`Seat \${String.fromCharCode(65 + row)}\${col + 1} is already taken.`); } } function updateSeatStatus(row, col, status) { const seats = document.getElementsByClassName('seat'); const index = row * 3 + col; seats[index].classList.remove('available', 'booked'); seats[index].classList.add(status); } function bookRandomSeat() { const availableSeats = []; for (let row = 0; row < theaterSeats.length; row++) { for (let col = 0; col < theaterSeats[row].length; col++) { if (theaterSeats[row][col] === 'O') { availableSeats.push({ row, col }); } } } if (availableSeats.length > 0) { const randomIndex = Math.floor(Math.random() * availableSeats.length); const { row, col } = availableSeats[randomIndex]; bookSeat(row, col); } else { alert('All seats are already booked.');</pre>
Classes	Classes são uma forma de criar modelos ou templates para objetos. Elas definem a estrutura e o comportamento dos objetos daquela classe.	<pre>class Person { constructor(firstName, lastName) { this.firstName = firstName; this.lastName = lastName; } getFullName() { return `\${this.firstName} \${this.lastName}`; } } // Creating an instance of the Person class const person1 = new Person("John", "Doe"); console.log(person1.getFullName()); // Output: "John Doe"</pre>
Objetos Construtores	Objetos são instâncias de classes ou podem ser criados como objetos independentes sem uma classe. Eles podem ter propriedades e métodos.	<pre>class Car { constructor(make, model, year) { this.make = make; this.model = model; this.year = year; } startEngine() { console.log(`The \${this.make} \${this.model}'s engine is running.`); } } const myCar = new Car("Toyota", "Camry", 2022); myCar.startEngine(); // Output: "The Toyota Camry's engine is running."</pre>

Literais de Objeto	Literais de objeto são uma forma de criar objetos ad-hoc sem definir uma classe.	<pre>const person = { firstName: "Alice", lastName: "Johnson", getFullName: function() { return `\${this.firstName} \${this.lastName}`; } }; console.log(person.getFullName()); // Output: "Alice Johnson"</pre>
Construtor de Função	Um construtor de função é uma função JavaScript regular que é usada para criar e inicializar objetos. É uma convenção nomear construtores de função com uma letra maiúscula inicial.	<pre>function Car(make, model) { this.make = make; this.model = model; } const car1 = new Car("Toyota", "Camry"); const car2 = new Car("Honda", "Civic"); console.log('Car1 details are', car1); console.log('Car2 details are', car2);</pre>
Notação de Ponto	A notação de ponto é uma forma de acessar propriedades de objetos.	<pre>const person = { firstName: "John", lastName: "Doe", age: 30 }; console.log(person.firstName); // Output: "John" console.log(person.lastName); // Output: "Doe" console.log(person.age); // Output: 30</pre>
Notação de Colchetes	A notação de colchetes é uma maneira de acessar propriedades de objetos, especialmente útil quando os nomes das propriedades contêm caracteres especiais ou espaços.	<pre>const person = { "first name": "John", "last name": "Doe", age: 30 }; console.log(person["first name"]); // Output: "John" console.log(person["last name"]); // Output: "Doe" console.log(person["age"]); // Output: 30</pre>
Arrays de Objetos	Um array de objetos em JavaScript é uma coleção de múltiplos objetos armazenados dentro de um único contêiner de array.	<pre>const students = [{ name: "Alice", age: 25 }, { name: "Bob", age: 22 }, { name: "Charlie", age: 28 }];</pre>

Acessar Array de Objetos	Você pode acessar elementos dentro de um array de objetos usando o índice do array e a notação de ponto.	<pre>const students = [{ name: "Alice", age: 25 }, { name: "Bob", age: 22 }, { name: "Charlie", age: 28 }]; console.log(students[0].name); // Output: "Alice" console.log(students[2].age); // Output: 28</pre>
Iterando Através de um Array de Objetos	A iteração de objetos através de arrays inclui loops for e métodos de array.	<pre>const students = [{ name: "Alice", age: 25 }, { name: "Bob", age: 22 }, { name: "Charlie", age: 28 }]; for (let i = 0; i < students.length; i++) { console.log(students[i].name); }</pre>
Adicionando Objetos	Você pode adicionar novos objetos ao array usando o método push.	<pre>//Adding Elements const students = [{ name: "Alice", age: 25 }, { name: "Bob", age: 22 }, { name: "Charlie", age: 28 }]; students.push({ name: "David", age: 20 }); // Add a new student console.log('After using push method '); console.log(students);</pre>
Removendo Objetos	Você pode remover objetos usando o método pop.	<pre>//Removing Elements const students = [{ name: "Alice", age: 25 }, { name: "Bob", age: 22 }, { name: "Charlie", age: 28 }]; const removedStudent = students.pop(); // Remove the last student console.log('After using pop method '); console.log(students);</pre>
Filtrando e Mapeando Arrays de Objetos	Você pode filtrar e transformar arrays de objetos usando métodos de array como filter e map.	<pre>const students = [{ name: "Alice", age: 25 }, { name: "Bob", age: 22 }, { name: "Charlie", age: 28 }]; const adults = students.filter(student => student.age >= 23); // Filter students who are 18 or oldercc const studentNames = students.map(student => student.name); // Create an array of student names console.log('Using Filter Method'); console.log(adults); console.log('Using Map Method'); console.log(studentNames);</pre>

Mapeando Arrays de Objetos	Você pode percorrer e transformar arrays de objetos usando métodos de array como map.	<pre>const employees = [{ name: "Alice", age: 35 }, { name: "Bob", age: 32 }, { name: "Charlie", age: 38 }]; const employee = employees.map((employee) => { return employee}); console.log(employee);</pre>
Buscando Objetos	Você pode buscar objetos dentro de um array de objetos usando métodos de array como find.	<pre>const employees = [{ name: "Alice", age: 35 }, { name: "Bob", age: 32 }, { name: "Charlie", age: 38 }]; const employee = employees.find(employee => employee.name === "Charlie"); console.log(employee.age);</pre>
Array Aninhado de Objetos	Um array de objetos é usado para armazenar e organizar dados de uma maneira que permite acessar e manipular as informações facilmente.	<pre>let arrayOfObjects = [{ name: 'John', age: 25, hobbies: ['Reading', 'Traveling'], address: { street: '123 Main St', city: 'New York', zip: '10001' } }, { name: 'Alice', age: 30, skills: ['JavaScript', 'React', 'Node.js'], projects: [{ title: 'Project A', completed: true }, { title: 'Project B', completed: false }] }, { title: 'Special Object', data: [1, 2, 3], metadata: { key: 'value' } }, { // An object with no specific properties }, { anotherObject: true, nestedArrays: [[1, 2, 3], ['a', 'b', 'c']], additionalProperty: 'Extra' }];</pre>
Acessar Array Aninhado - Código Acima	Usando o operador de ponto, os elementos do array aninhado podem ser acessados, o que foi descrito no código acima.	<pre>// Accessing properties of the first object console.log(arrayOfObjects[0].name); // Output: John console.log(arrayOfObjects[0].hobbies[0]); // Output: Reading // Accessing properties of the second object console.log(arrayOfObjects[1].skills[2]); // Output: Node.js console.log(arrayOfObjects[1].projects[0].title); // Output: Project A // Accessing properties of the third object console.log(arrayOfObjects[2].metadata.key); // Output: value // Accessing properties of the fourth object console.log(arrayOfObjects[3]); // Output: {} // Accessing properties of the fifth object console.log(arrayOfObjects[4].anotherObject); // Output: true console.log(arrayOfObjects[4].additionalProperty); // Output: Extra</pre>

Strings	Strings são um tipo de dado em JavaScript usados para representar texto. Eles podem conter letras, números, símbolos e caracteres de espaço em branco.	<pre>const message = "This is a message.";</pre>
Strings	Strings são um tipo de dado em JavaScript usado para representar texto. Eles podem conter letras, números, símbolos e caracteres de espaço em branco.	<pre>const message = "This is a message.";</pre>
literais de template	Literais de template em JavaScript são strings que permitem expressões embutidas, denotadas por crases (), permitindo strings de várias linhas e interpolação de variáveis usando <code>\${}</code> .	<pre>const fullName = `\${firstName} \${lastName}`;</pre>
Concatenação de Strings	O operador de concatenação + em JavaScript é usado para combinar (unir) duas ou mais strings para criar uma única string mais longa.	<pre>const firstName='Peter'; const greeting = 'Hello, ' + firstName + '!'; console.log(greeting);</pre>
Comprimento da String	Para determinar o comprimento de uma string, a propriedade <code>length</code> pode ser utilizada.	<pre>const message1 = "This is a message."; const Stringlength1 = message1.length; const message2 = "Thisisamessage"; const Stringlength2 = message2.length; console.log(Stringlength1); console.log(Stringlength2)</pre>
Acessando Caracteres	Caracteres individuais dentro de uma string podem ser acessados usando a notação de colchetes e um índice baseado em zero.	<pre>const text = "JavaScript"; const firstCharacter = text[0];</pre>
toLowerCase e toUpperCase	JavaScript fornece métodos para alterar o caso de uma string	<pre>const text = "Hello, World!"; const lowercaseText = text.toLowerCase(); // "hello, world!" const uppercaseText = text.toUpperCase(); // "HELLO, WORLD!" console.log('The lowercase for text is ',lowercaseText);</pre>

	para minúsculas e maiúsculas.	<pre>console.log('The uppercase for text is ',uppercaseText);</pre>
método indexOf()	indexOf retorna o índice da primeira ocorrência de uma substring especificada dentro de uma string. Retorna -1 se a substring não for encontrada.	<pre>const sentence = "The quick brown fox jumps over the lazy dog."; const indexOfFox = sentence.indexOf("fox"); // 16 console.log(indexOfFox);</pre>
método includes()	includes retorna um booleano indicando se uma substring especificada é encontrada dentro de uma string, retornando true se encontrada e false se não.	<pre>const sentence = "The quick brown fox jumps over the lazy dog."; const hasFox = sentence.includes("fox"); // true console.log(hasFox);</pre>
métodos substring()	substring extrai caracteres de uma string entre dois índices especificados. Isso significa extrair uma substring do texto começando no índice 0 e terminando no índice 5 (excluindo o índice 5).	<pre>const text = "Hello, World!"; const subText1 = text.substring(0, 5); // "Hello" console.log(subText1);</pre>
método slice()	slice extrai uma seção de uma string e a retorna como uma nova string, especificando as posições de início e fim. Isso significa extrair uma substring do texto começando no índice 7 até o final da string.	<pre>const text = "Hello, World!"; const subText2 = text.slice(7); // "World!" console.log(subText2);</pre>
método substr()	substr extrai um número especificado de caracteres de uma string, começando em um índice especificado. Isso significa extrair uma substring do texto começando no 7º índice e incluindo 5 caracteres.	<pre>const text = "Hello, World!"; const subText3 = text.substr(7, 5); // "World" console.log(subText3);</pre>
Substituindo Substrings	O método replace permite substituir substrings por novos valores.	<pre>const text = "Hello, World!"; const updatedText = text.replace("World", "Universe"); console.log(updatedText);</pre>

Dividindo Strings	Você pode dividir uma string em um array de substrings usando o método split.	<pre>const csvData = "Alice,25,New York;Bob,30,Los Angeles;Charlie,28,Chicago"; const peopleArray = csvData.split(';'); console.log(peopleArray);</pre>
método trim()	O método trim remove espaços em branco no início e no final de uma string.	<pre>const text = " Trim me! "; console.log(text.length); const trimmedText = text.trim(); console.log(trimmedText.length);</pre>
métodos matemáticos round(), ceil() e floor()	round() arredonda um número para o inteiro mais próximo. ceil() arredonda um número para cima até o inteiro mais próximo. floor() arredonda um número para baixo até o inteiro mais próximo.	<pre>const number = 3.6; const rounded = Math.round(number); // Round to nearest integer: 4 const ceil = Math.ceil(number); // Round up: 4 const floor = Math.floor(number); // Round down: 3</pre>
Métodos Matemáticos pow(), sqrt() e log()	pow() eleva um número a um expoente especificado. sqrt() retorna a raiz quadrada de um número. log() retorna o logaritmo natural (base e) de um número.	<pre>const base = 2; const exponent = 3; const power = Math.pow(base, exponent); // Power: 8 const squareRoot = Math.sqrt(base); // Square Root: 1.41421356237 const naturalLog = Math.log(base); // Natural Logarithm: 0.69314718056</pre>
Método random()	O método random() em JavaScript gera um número de ponto flutuante pseudo-aleatório entre 0 (inclusive) e 1 (exclusive).	<pre><!DOCTYPE html> <html> <head> <title>Random Quote Generator</title> </head> <body> <h1>Random Quote Generator</h1> <p id="quoteDisplay"></p> <button onclick="generateRandomQuote()">Get Quote</button> <script> const quotes = ["Life is what happens when you're busy making other plans. - John Lennon", "The only way to do great work is to love what you do. - Steve Jobs", "In three words, I can sum up everything I've learned about life: it goes on. - Robert Frost", "Don't count the days, make the days count. - Muhammad Ali", "The only thing we have to fear is fear itself. - Franklin D. Roosevelt", "To be yourself in a world that is constantly trying to make you something else is the greatest :]; function generateRandomQuote() { const randomIndex = Math.floor(Math.random() * quotes.length); // Generate a random index const randomQuote = quotes[randomIndex]; // Get a random quote document.getElementById("quoteDisplay").textContent = randomQuote; } </script> </body> </html></pre>

Objeto Date	Objetos Date são usados para representar momentos específicos no tempo.	<pre>const currentDate = new Date(); // Current date and time const specificDate = new Date(2023, 0, 15); // January 15, 2023 const fromMilliseconds = new Date(1672569600000); // From milliseconds since the epoch</pre>
Data de Recuperação	Objetos de data fornecem acesso a componentes individuais de uma data, como ano, mês, dia e hora.	<pre>const date = new Date(); const year = date.getFullYear(); // Current year const month = date.getMonth(); // Current month (0-11) const day = date.getDate(); // Day of the month (1-31) const hours = date.getHours(); // Hours (0-23) const minutes = date.getMinutes(); // Minutes (0-59) const seconds = date.getSeconds(); // Seconds (0-59)</pre>
toLocaleDateString() e toLocaleTimeString()	toLocaleDateString() converte uma data em uma string que representa a parte da data de acordo com as convenções de formatação da localidade. toLocaleTimeString() converte uma data em uma string que representa a parte do tempo de acordo com as convenções de formatação da localidade.	<pre>const date = new Date(); const formattedDate = date.toLocaleDateString(); // "11/15/2023" const formattedTime = date.toLocaleTimeString(); // "1:30:45 PM"</pre>
Aritmética de Datas	Objetos de data permitem várias operações de aritmética de datas, incluindo a adição e subtração de intervalos de tempo.	<pre>const date = new Date(); date.setFullYear(2024); // Set the year to 2024 date.setDate(date.getDate() + 7); // Add 7 days const futureDate = new Date(); futureDate.setDate(futureDate.getDate() + 30); // Date 30 days from now</pre>
Método setTimeout()	A função setTimeout agenda a execução de uma função após um atraso especificado em milissegundos:	<pre>setTimeout(function() { console.log("This message appears after a delay."); }, 2000); // Displayed after a 2-second delay</pre>
setInterval	setInterval executa repetidamente uma função em um intervalo especificado.	<pre>let count = 0; const intervalId = setInterval(function() { console.log("Count: " + count); count++; if (count > 5) { clearInterval(intervalId); // Stop after 6 iterations } }, 1000); // Displayed every second.</pre>



Skills Network